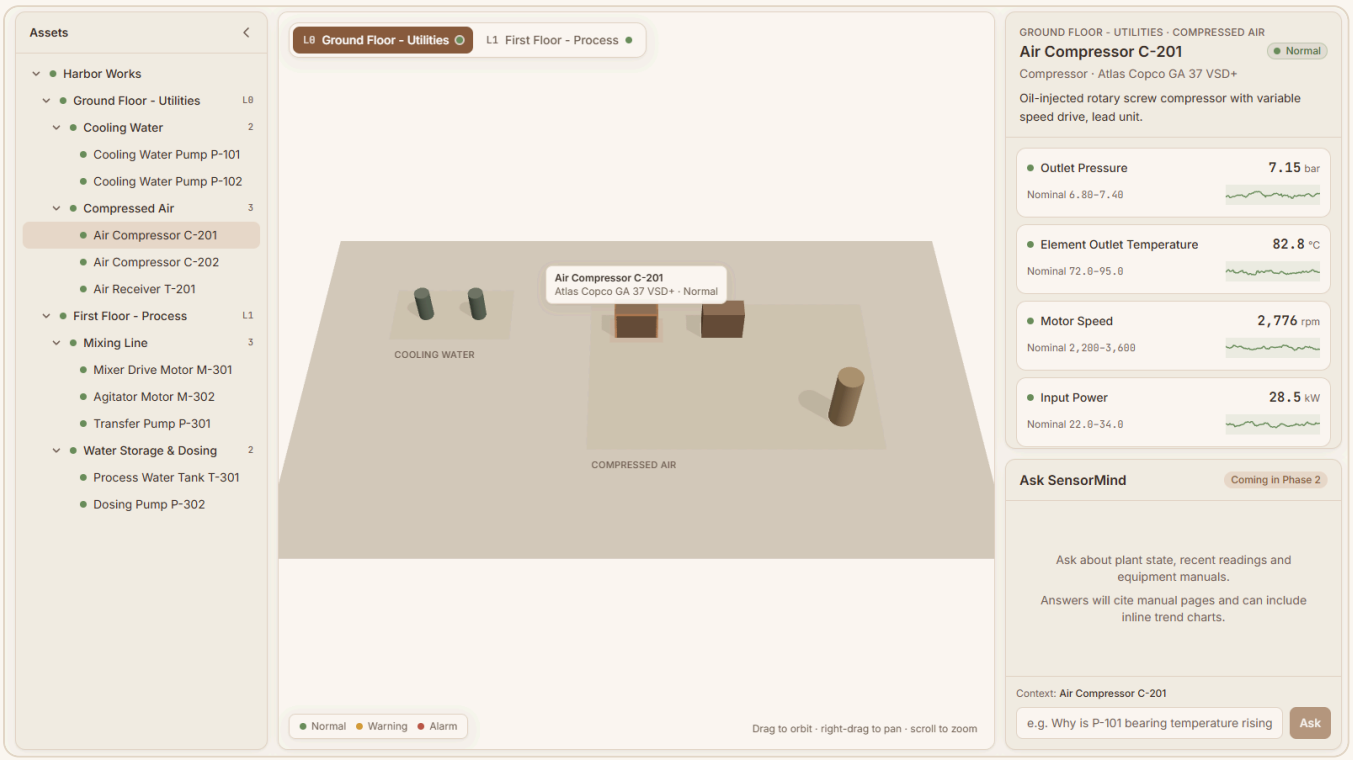
SensorMind

An industrial IoT platform that links live sensor readings to a 3D view of the plant, with a planned assistant that answers questions from equipment manuals.

Ali Qumail · github.com/AliQumail/SensorMind · MIT licensed



Selecting Air Compressor C-201 in the tree or the 3D scene brings up its live tags, nominal bands and recent trends. All readings are simulated.

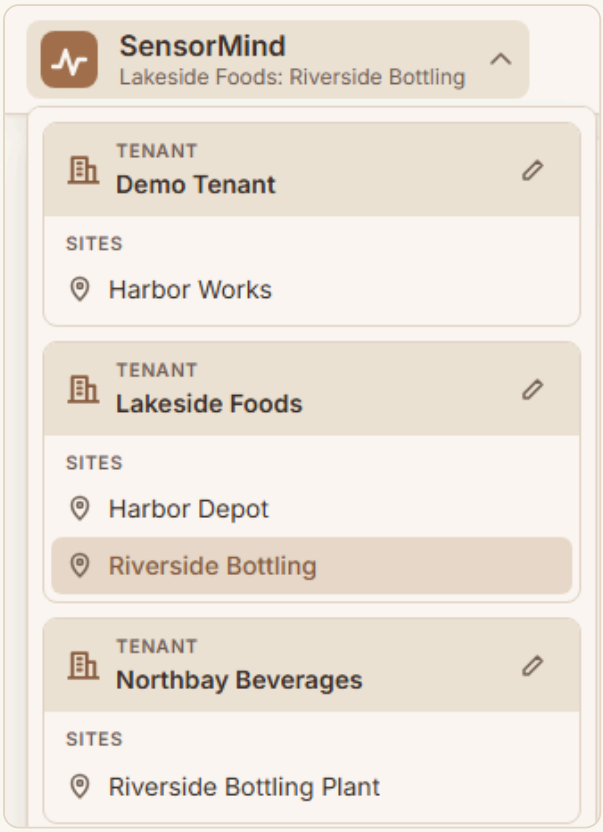
2 s live sampling, streamed to the browser	**36** sensor tags on 10 assets in the sample plant	**60** automated backend tests	**~5,400** lines of C# and TypeScript

The idea

A sensor reading only means something in context: which machine it belongs to, where that machine sits, and what its normal range is. SensorMind brings those together in one screen. An engineer can move from a site to a floor, a machine and its sensors, and see at once whether anything is outside its normal band.

What it does

The hierarchy runs Tenant → Site → Floor → Group → Equipment → Tag. Everything below the tenant is defined in configuration, so a new plant needs no code changes.



The tenant menu separates tenants from their sites and gives each tenant an edit button.

Live monitoring

A browser-based 3D floor view, an asset tree and a sensor panel stay in sync. Each tag is rated normal, warning or alarm against its configured limits, and the worst status rolls up through the tree and tints the machine in 3D.

Multi-tenant management

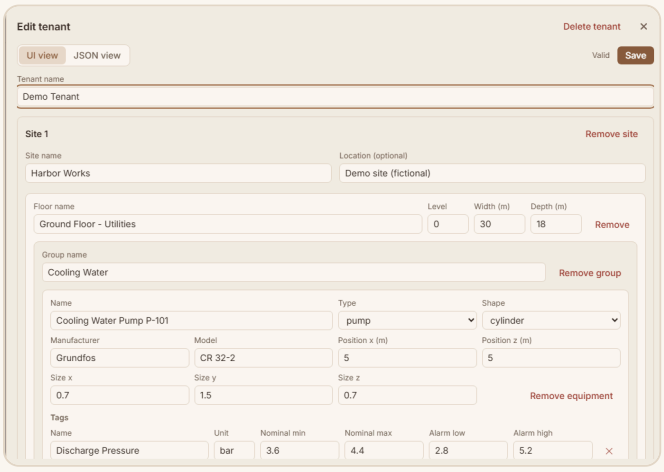
After signing in, an administrator can add, edit or delete a tenant from the menu. Saving takes effect straight away: the simulator starts on new sites and stops on removed ones without a restart.

Realistic simulated data

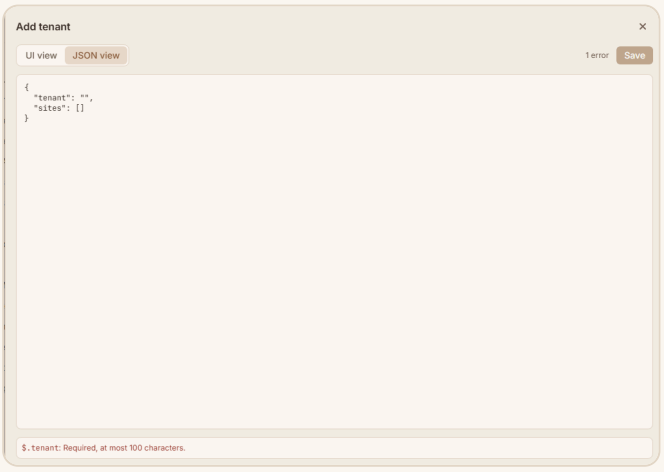
Each tag follows a mean-reverting random walk with occasional drifts, spikes and alarm excursions. A fixed seed makes runs repeatable, and 24 hours of history is generated at startup so charts have data immediately.

One document, two views

A tenant and everything under it is a single JSON document. It can be edited through forms, with starter templates for common equipment, or as raw JSON. Both views edit the same document and use the same server-side validator, which reports every error with its exact JSON path.



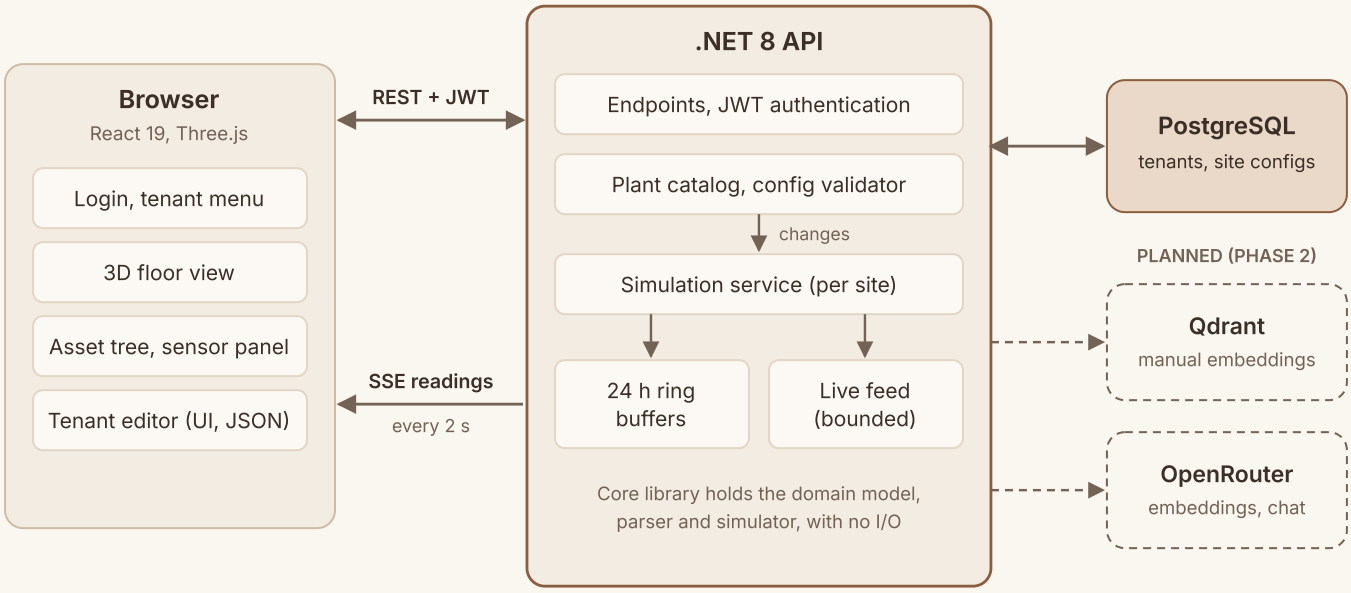
UI view: forms for sites, floors, equipment and tags.



JSON view: a new tenant starts empty, and Save stays off until it is valid.

How it is built

A React single-page app talks to a .NET 8 API. Requests use REST with a JSON Web Token, and live readings arrive over Server-Sent Events. Domain logic sits in a core library with no I/O, so it can be tested without a server or database.



Key engineering decisions

DECISION	WHY
Report all config errors at once	Validation collects every problem with a JSON path, and unknown properties count as errors. A typo such as `nominalMaxx` cannot pass unnoticed.
Server-Sent Events, not WebSockets	Readings flow one way. SSE reconnects by itself and works as plain HTTP. Each client has a bounded buffer, so a slow one cannot hold up the others.
Deterministic simulator	Each tag has its own random generator derived from the seed and its id, so adding a tag does not change the values of any other tag.
Ring buffers behind an interface	Memory stays fixed at about 25 MB for 24 hours of the sample plant, and a real time-series database can replace them later.
Tenant saved in one transaction	A failed save leaves nothing half written. Site and tag ids are checked across all tenants first, because live data is keyed by tag id.
Status calculated on the server	Threshold logic lives in one place. The UI only rolls statuses up, so the tree, panel and 3D scene cannot disagree.
Measured colour contrast	Text colours were checked against WCAG AA and adjusted where they failed. Status also appears as labelled badges, not colour alone.

Status and roadmap

STAGE	STATE	SCOPE
Skeleton	Done	Config loader and validator, asset tree, 3D floor view, simulated stream, live sensor panel
Tenant management	Done	Login, tenants and sites, single-document editor with UI and JSON views, equipment templates
PostgreSQL migration	In progress	Code switched from SQLite and compiling. Still to be verified against a running database.
RAG assistant	Next	PDF ingestion, Qdrant, embeddings, streaming chat with citations to manual pages
Intelligence	Planned	Selected-asset context, live readings in prompts, inline trend charts through tool calling
Hardening	Planned	Rate limiting, bring-your-own key, continuous integration, public deployment

How it was tested

- 60 xUnit tests cover the config parser, simulator, status rules, sensor store, live feed and downsampling.
- Adding, editing and deleting tenants was run end to end in a scripted browser with no console errors. That run used the SQLite build, before the PostgreSQL switch.
- Release builds treat compiler and analyzer warnings as errors.

Current limitations

- All sensor data is simulated. No real devices are connected.
- There is one demo admin account with a default password, so this is not ready for production.
- Tenants are not yet isolated: any signed-in user can see all of them.
- Tenant storage and editing have no automated tests yet, and there is no public deployment.

Technology

Backend	.NET 8 Minimal APIs, JWT bearer authentication, Npgsql, xUnit
Frontend	React 19, TypeScript, Vite, Three.js with React Three Fiber, Recharts, Tailwind CSS 4
Data	PostgreSQL 17 for configuration, in-memory ring buffers for readings
Planned	Qdrant for vector search, OpenRouter for embeddings and chat

Source code

Code, setup steps, API reference and the full decision log are at `github.com/AliQumail/SensorMind`.